Faculté des Sciences de Tunis
Département des Sciences de l'Informatique

Documents non autorisés

Session Principale : Mai 2024
Épreuve : Systèmes & Applications Répartis
Section : IGL3 Durée : 1H30

Nombre de pages : 1

Enseignants : H. Abbes & I. Denden

QCM (5 points)

1. Quelle est la fonction principale de l'objet UnicastRemoteObject dans Java RMI ?
   a. Gérer les appels synchrones
   b. Activer le garbage collection
   c. Faciliter l'exportation d'objets distants
   d. Créer des connexions réseau sécurisées

2. Quel langage de définition est utilisé pour définir les services et les messages dans gRPC ?
   a. JSON
   b. YAML
   c. Protocol Buffers
   d. XML

3. Quel code de statut HTTP est généralement utilisé pour indiquer qu'une ressource a été créée avec succès ?
   a. 200
   b. 201
   c. 204
   d. 400

4. Parmi les méthodes HTTP suivantes, lesquelles sont considérées comme idempotentes ?
   a. GET
   b. POST
   c. PUT
   d. DELETE

5. Que fait le code suivant dans un serveur gRPC en Java ?

```
public class GreeterImpl extends GreeterGrpc.GreeterImplBase {
  @Override
  public void sayHello(HelloRequest req, StreamObserver<HelloReply> responseObserver) {
    HelloReply reply = HelloReply.newBuilder().setMessage("Hello " + req.getName()).build();
    responseObserver.onNext(reply);
    responseObserver.onCompleted();
  }
}
public static void main(String[] args) throws IOException, InterruptedException {
  Server server = ServerBuilder.forPort(50051)
    .addService(new GreeterImpl())
    .build()
    .start();
  server.awaitTermination();
}
```

   a. Définit un service gRPC qui répond avec "Hello" suivi du nom fourni dans la requête.
   b. Lance un serveur HTTP sur le port 50051.
   c. Enregistre un service RESTful sur le port 50051.
   d. Affiche "Server started, listening on 50051" sans démarrer aucun service.

Scanned with CamScanner

Exercice 1 (12 points)

Créer un service de calculatrice simple utilisant Java RMI. Le service devra être capable de réaliser des opérations de base : addition, soustraction, multiplication, et division.

1. Définir l'Interface Distante Calculator
2. Implémenter l'Interface Distante CalculatorImpl
3. Créer le Serveur RMI en utilisant LocateRegistry() et le port 1099.
4. Créer le Client RMI [redacted] sans utiliser LocateRegistry()
5. Donner toutes les instructions pour compiler et lancer le service, serveur et client.
6. Donner les endpoints équivalents si on veut exposer le service en restfull sur le serveur calculator.fst.utm.tn

Exercice 2 (3 points)

1. Compléter la ligne du code qui permet d'envoyer un message au client

```
public class SimpleServer {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(12345)) {
            Socket clientSocket = serverSocket.accept();
            PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true);
        } catch (IOException e) {
            e.printStackTrace();
}}
```

2. Compléter la ligne du code qui permet de lire un message envoyé par le serveur

```
public class SimpleClient {
    public static void main(String[] args) {
        try (Socket socket = new Socket("localhost", 12345);
            BufferedReader in = new BufferedReader(new InputStreamReader(socket.getInputStream())))
            {
                System.out.println("Message from server: " + message);
            } catch (IOException e) {
                e.printStackTrace();
}}}
```

3. Analysez le code suivant et identifiez l'erreur.

```
public class ErrorServer {
    public static void main(String[] args) {
        try (ServerSocket serverSocket = new ServerSocket(12345)) {
            while (true) {
                Socket clientSocket = serverSocket.accept();
                PrintWriter out = new PrintWriter(clientSocket.getOutputStream(), true)
                clientSocket.close();
            }
        } catch (IOException e) {
            e.printStackTrace();
}}}
```

Scanned with CamScanner